

birdclef-2026

1位解法: Noisy Student と Distillation の融合

Rank	1
Team	Nikita Babych
Author	Nikita Babych
Topic ID	704752
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/birdclef-2026/discussion/704752>

Retrieved with Kaggle CLI on 2026-07-03.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

原文（Kaggle Discussion）：<https://www.kaggle.com/competitions/birdclef-2026/discussion/704752>

お祝いと感謝

入賞された皆さん、おめでとうございます！特に4,000チームと競い合う状況では、どれほどの労力がかかるかわかります。毎日のように新しい共有ノートブックが登場し、それらは往々にして過学習しているにもかかわらず、より凝った複雑な手法でも打ち負けず落胆させられるのですから。

そしていつものように、今年もまた BirdCLEF で競わせてくれた Kaggle と主催者の皆さんに感謝します。そして、BirdCLEF の歴史に新しい要素 — Validation（検証）!! — を導入してくれたことにも感謝します。さらに、Discussions や Notebooks で興味深いアイデアを共有してくれた Kaggle コミュニティにも感謝しなければなりません。その中のいくつかは、私も実際に取り入れました。

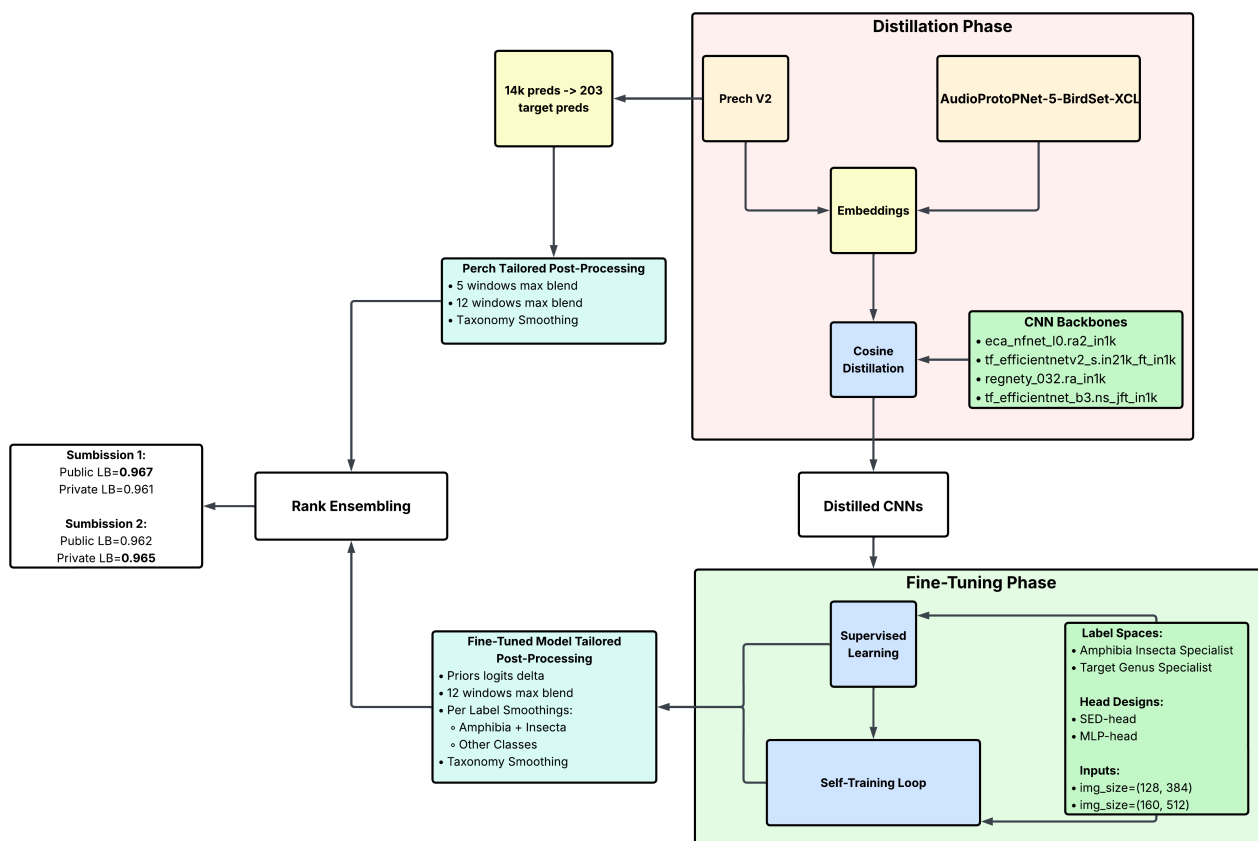
はじめに

初日からこのコンペは過酷なものに感じられました。昨年のアイデアはどれも競争力のある結果を生まず、私が旧来のやり方で地道にコーディングしている間に、他の参加者はコーディングエージェントで開発を自動化してリーダーボードを駆け上がっていくのを目の当たりにしていました。テストしたいアイデアがたまっており、追いつきたいという強い思いから、私は月額100ドルの Claude Code サブスクリプションを購入し、そのドキュメントとコースをひとつとおこなしました。ほどなくして、私ははるかに速くイテレーションを回せるようになり、LB でも堅実な結果を出せるようになりました。

とはいえ、私は強い持論を持っています。SOTA のエージェントはソフトウェア開発には優れていますが、新規性が問われる DS/ML では相対的に弱いということです。したがって、Kaggle の作業は2つの部分に分かれると私は主張したいのです。1つ目はソフトウェアで、これはコードを書くという大部分が機械的なプロセスであり、エージェントが見事に自動化してくれます。2つ目は DS/ML で、ここではアイデアを生み出し（LLM が学習済みの内容を再現するだけでなく）、自分の結果から直感を養い、何かオリジナルなものにたどり着かなければなりません。そうした洞察は、どんなに小さなものでも競争力を持つのに十分たり得ます。この2つ目の部分では、Web を近似するのではなく、時折本当に良いアイデアを閃かせてくれる「非線形性」を人間のエンジニアは持っているため、完全自動化されたエージェントよりも人間の方に分があると私は信じています（もっとも、単に私がエージェントを十分に使いこなせていないだけかもしれませんが）。

解法の概要

この解説を読む前に、まず**私の昨年の解説**に目を通しておくことを強くお勧めします。そうすることで、今回の解法が築かれている土台の文脈を読者が把握できるからです。そうでなければ、いくつかの要素をここで全面的に再掲しなければならないになりますが、それらを改めて説明するのは冗長だと判断しました。



略語

- **LSS**: labeled soundscapes (ラベル付きサウンドスケープ)
- **USS**: unlabeled soundscapes (ラベルなしサウンドスケープ)
- **PL**: pseudo-labels (擬似ラベル)
- **MLP**: multilayer perceptron (多層パーセプトロン)
- **SED**: sound event detection (音響イベント検出、attention-pooling ヘッド)
- **TTA**: test-time augmentation (テスト時拡張)
- **LB**: leaderboard (リーダーボード)

TLDR

- **5秒入力**による多様なアンサンブル: SED ヘッド CNN、MLP ヘッド CNN、Amphibia/Insecta (両生類/昆虫) スペシャリスト、genus (属) レベルのスペシャリスト、そして Perch v2 (線形ヘッド、ファインチューニングなし)。
- 2つの学習フェーズ: cosine embedding distillation (コサイン埋め込み蒸留、Perch v2 [1] から、1モデルは AudioProtoPNet-5-BirdSet-XCL [2] から) と、full fine-tuning (教師あり学習、その後 Multi-Iterative Noisy Student による自己学習)。
- 昨年の解法の自己学習アプローチにいくつかの調整を加えたもの: 注入するラベル和を正規化し、LSS を別の injector として追加し、Site 22 では非 LSS 種をマスクする。
- ドメインに合わせて仕立てた2つの LSS スプリットでの検証: 未知サイトへの汎化を確認する Site-22 スプリットと、種のカバレッジを最大化する greedy スプリット。

- パイプラインごと（ファインチューニング済みモデル vs. Perch ネイティブ）にチューニングした後処理。これは LB を探る（probing）のではなく、主に検証スプリット上で行った。
- Perch v2 と私のモデルの予測との間の Rank Blending（ランクブレンディング）。
- AI ツール: DS エージェント = 私自身、ソフトウェアエージェント = Claude Code。

データ

追加データ

対象種について [Xeno-Canto](#) や [iNaturalist](#) から引っ張ってきた追加データは、どれも結果を一貫して悪化させたため、学習にはコンペのデータのみを使用しました:

- Train focal data（フォーカルの学習データ）
- LSS
- USS

唯一の例外は Amphibia/Insecta スペシャリストです。これには、私が昨年共有した追加 Xeno-Canto 種の [データセット](#) を、iNaturalist のサンプルと新規種で拡充したものを使いました:

- 41,127 サンプル
- 1,903 のユニーク種

Focal データの準備

- 音声を **5秒チャンク** にスライスしました。今年は5秒より長いモデルはうまく機能せず、私はこれを2通りに説明します:
 - **ドメイン適応（Domain adaptation）**。私たちの主なドメイン適応メカニズムは、テストドメインの LSS を注入することでした。LSS には、ノイズの多い環境で一度だけ出現するような非常に稀な種が含まれます。ラベル付けされた5秒チャンクを超える文脈を取り除くことで、モデルにどこに注目すべきかを伝えることになり、それが役立ちました。
 - **鳴き声とラベル付けのパターン（Call and labeling patterns）**。最適な長さは、鳴き声が長い種と短い種の構成比、それらをそのドメインで分離することの難しさ、そしてアノテーターがどうラベル付けするか（たとえば、ある年は5秒チャンクとのわずかな鳴き声の重なりでもカウントするか否か）によって、年ごとに変化します。（この長さの探索は、実運用システムにおいてはドメインごとに自動化できるのではないかと私は推測しています。）
- 各スライス は abs-max （絶対値の最大）で正規化されます。

LSS の準備

昨年の injector のコンセプト（もともとは PL 用に作ったもの）を再利用しました: 各バッチの一部について、ランダムな LSS チャンクを mixup で背景に混ぜ込み、フォーカルサンプルのラベルと、サンプリングした5秒 LSS チャンクのラベルとの要素ごとの max を取ります。

LSS サンプルがこれほど少ないため、このアプローチはバッチあたり数回の注入でも急速に過学習していました。これを、各ラベルに1を割り当てる代わりにサンプリングした LSS のラベル和を 0.5 に正規化することで修正しました。これにより、それらの種への学習シグナルを最小化し、フォーカルのみに存在する種に注力できるようになります。これを行った後、かなりうまく機能し始めました。

ほとんどの学習で使用した LSS-injector のパラメータ:

- バッチあたりの LSS 混合サンプルの割合: **0.25 から 0.75** (学習フェーズによる)。
- バッチあたりのクリーンな (連結された、非混合の) LSS チャンク: **2**。テストドメインのクリーンなサンプルをいくつか見せるため。
- ラベル和: **0.5**。
- その他の準備はすべて focal データと同じ。

検証 (Validation)

LSS サンプルはそれほど多くないため、以下の2つの補完的なスプリットを用いて LSS 上で検証を行い、それらのスコアを平均しました:

- **Site-22 split**。Site 22 のサンプルをすべて train に移し、残りで検証する。これは純粹に未知サイトへの汎化を確認するためです。
- **Greedy split**。各種のサンプルを train に少なくとも1つ残しつつ、できるだけ多くの種を validation に押し込み、メトリクスの種ごとのカバレッジを最大化する。

常に強い相関が見られたとは言えませんが、シグナルは安定しており、学習と後処理のパラメータを十分うまくチューニングできる程度には良好でした。たとえば、検証を用いてたどり着いた学習パラメータは、結果的に私が得た中で最良のものであり、LB 向けに特別に再調整しようとしたときよりもさらに良いものでした。

学習 (Training)

蒸留 (Distillation)

まず、昨年の解法に倣い、Perch も事前学習も一切使わずにモデルを学習させようと思いました。というのも、私はインドメインの自己学習の力を信じており、それがあらゆるものを打ち負かせると考えていたからです。しかし、2回のイテレーションの自己学習アンサンプル (LB=0.931) が、Perch のみを使ったエージェント作成の共有ノートブックよりも劣ることを見て、かなり早い段階で落胆しました。そこで私はこのクラブのルールを受け入れ、Perch の蒸留を試し始め、私にとって非常にうまく機能する2フェーズ学習にたどり着き、自己学習なしで LB=0.935+ を達成できました。

1. **バックボーンを蒸留する**。cosine loss を用い、線形の射影ヘッドを介して教師の embeddings から蒸留する。
2. **蒸留を無効化し**、蒸留済みモデルに対して低い LR で標準的なエンドツーエンドのファインチューニングを行う (mlp モデルについては、ファインチューニング中も蒸留をアクティブにした)。

私はすべてのモデルをこの方法で学習させ、まずバックボーンを蒸留し、その後、該当する PL やスペシャリストの設定でファインチューニングを行い、多様性のために1つのモデルでは Perch を AudioProtoPNet-5-BirdSet-XCL に差し替えました。

1点、注記しておくべき詳細があります。異なるヘッド/ラベル設計で同じバックボーンを再利用する場合でも、私は**毎回バックボーンを再蒸留しました**。蒸留 loss は非ゼロのままなので、これがモデル間にわずかな多様性を加えます。たとえば、異なる seed で蒸留した以外は同一の2モデルをアンサンプルすると、顕著なブーストが得られました。

蒸留 loss については、正規化なしの MSE よりも速く、より良く収束したため、**cosine** に落ち着きました。

蒸留フェーズ

すべてのモデルの最適な学習パラメータは非常に似通っていたため、以下の設定で学習しました:

Parameter	Value
Epochs	11
LR	5e-4
Loss	cosine
Target	teacher embeddings, no processing
Scheduler	one-cycle cosine
Data	focal train + soundscapes
Mixup	p = 0.5
Optimizer	AdamW
Batch size	64

ファインチューニングフェーズ

この下流フェーズでは LR を 2e-4 に下げ、モデル間で epoch 数を少し変えました。教師ありファインチューニングと自己学習の間では、それ以外は何も変えていません。

Parameter	Value
Epochs	8 から 15 (Amphibia と Insecta のスペシャリストは 40)
LR	2e-4
Loss	CE
Scheduler	one-cycle cosine
Data	focal train + LSS + 擬似ラベル付き USS (有効な場合)
Target	正規化された species/genus ラベル
Mixup	p = 0.5
Optimizer	AdamW
Batch size	64

自己学習: 昨年のレシピを適応させる

昨年の自己学習レシピは、蒸留を一切使わずにモデルを学習させ、そのスコアが LB=0.925 未満だったときにはうまく機能しました。しかし、事前蒸留したモデルのファインチューニングに切り替えると、あらゆる PL が結果を PL なしの場合よりも**はるかに悪化**させました。いろいろ探った結果、原因を突き止めました。蒸留 + LSS がすでに非常に強力なベースを与えるため、たとえべき乗変換 (power transform) を施した後であっても、わずかにノイズのある PL を加えるだけで致命的になり、シグナル全体をそれら自身の方へ引き寄せてしまうのです。

第1の突破口は、**PL のラベル和を focal のラベル和より低くキャップ**することでした（LSS injector と同じメカニズムですが、動機は少し異なります）。これはテストドメインの背景を保ちつつ、そこに存在する可能性が高い種へのかすかなシグナルを加え、それらのノイズへの過学習を避けます。これを駆動した2つの考慮点があります：

- LSS はすでに一部の種について本物の ground truth を提供するので、よりノイズの多い PL はそうした種を悪化させるだけです（LSS に由来する種の擬似ラベルは非常にノイズが多かった）。
- LSS でカバーされない稀な種は、依然としてそのシグナルの大部分を focal データから得る必要がありますが、そこでは PL が信頼できないため、そのシグナルは抑制されなければなりません（私は各バッチで LSS を見せていたので、非 LSS 種は明らかに underfit してしまい、その値は LSS の種よりもはるかに低く、たいていはノイズでした）。

第2の突破口は、**同じバッチ内に LSS と擬似ラベル付き USS を注入**することでした。これは、LSS がすでによく教えている種について PL のノイズを相殺するためです。重要な機微: 2つの injector は**重複してはならない**ということです。バッチあたり固定数の focal サンプルを取り、そのうちのいくつかには LSS を、別のいくつかには擬似ラベル付き USS を注入し、同じサンプルに両方を注入することは決してしません。

そして、結果を少し（~0.002 の話です）改善するのに役立った最後の要素は、Site 22 について LSS に出現しなかったすべての種をマスクすることでした。これは、非 LSS 種はそこでは稀であると仮定できるだけの Site 22 サンプルが十分にあるという事前分布（prior）に従ったものです。驚いたことに、これが機能しました。

昨年のレシピの他の要素も重要でした。たとえば power transform や、最大ラベル和に応じたサンプリングなどです。

自己学習中、両方の injector は別々の focal サンプル上で同時に動作します：

Parameter	LSS injector	PL injector
Injection ratio per batch	0.1875	0.75
Clean concats per batch	2	-
Label sum cap	0.5	0.75

この設定で昨年の魔法が再びやってきましたが、有効なのは2イテレーションまででした。

以下は、smoothing 付きの eca_nfnet_l0.ra2_in1k SED モデルを2つの seed で提出して追跡した結果です：

Pseudo Iteration	LB score
1-stage only supervised learning	0.935
1-pseudo iteration	0.946
2-pseudo iteration	0.950
3-pseudo iteration	0.949

モデル

異なるヘッド、ラベル空間、入力を持つモデルを（さらに異なる CNN ファミリーの上で）アンサンブルしなければ、競争力のある結果は得られませんでした。理由は、蒸留 + 自己学習が高度に相関したモデルを生み出すため、さらに押し上げる唯一の方法が、設計の多様性を通じて非線形性を加えることだったからです。ブーストは大きくはありませんでしたが、public LB で1位に到達し、private でもそれを維持するには十分でした。

すべてのモデルの中で、より詳しく説明すべきなのは genus（属）レベルのモデルです。というのも、多くの種（特に稀な両生類）は過密なサウンドスケープの中では区別できなかったのですが、種間で通常は似ている主要な genus のシグナルを得るだけで十分なこともあり、それによって、他の方法では認識不能なそれらの種の予測に対して非常に有益なエンリッチメント（強化）を提供できたのです。そこで私は、データ準備の際に同じ genus に関連するすべてのラベルの最大値を取ることで（擬似ラベルも含めて）、種の代わりに genus を予測するようモデルを学習させ、同じ genus のすべての種に同じ値を広げてその予測をブレンドしました。LB=0.96+ で行き詰まっていたときに、~0.001-0.002 という非常に良いブーストが見られました。

最終アンサンブル:

Backbone	Head	Label space	Distilled from	Pseudo iters	Mel input
regnety_032.ra_in1k	SED	target	Perch v2	2	128×384
regnety_032.ra_in1k	SED	target	Perch v2	3	128×384
eca_nfnet_l0.ra2_in1k	SED	target	Perch v2	2	128×384
eca_nfnet_l0.ra2_in1k	SED	target	AudioProtoPNet-5-BirdSet-XCL	2	128×384
tf_efficientnetv2_s.in21k_ft_in1k	SED	target	Perch v2	1	160×512
tf_efficientnetv2_s.in21k_ft_in1k	SED	target	Perch v2	2	160×512
tf_efficientnet_b3.ns_jft_in1k	SED	extended (Amphibia + Insecta)	Perch v2	supervised only	128×384
eca_nfnet_l0.ra2_in1k	MLP	target	Perch v2	2	128×384
eca_nfnet_l0.ra2_in1k	SED	genus-level	Perch v2	2	128×384
Perch v2 native	Linear	203 overlapping targets	-	-	native frontend

メルスペクトログラムのパラメータ

Mel 入力は 32 kHz 音声、5秒クリップ、n_fft 2048、f_min 0、f_max 16000、power 1 を共有します（Perch v2 を除く）:

- 128×384 (n_mels 128, time 384) : hop ≈ 417。すべての eca および regnety モデル、および b3 スペシャルリストで使用。
- 160×512 (n_mels 160, time 512) : hop ≈ 313。tf_efficientnetv2_s モデルで使用。

後処理 (Postprocessing)

処理チェーンは、2つのパイプラインでキャリブレーションが異なるため、**ファインチューニング済みモデル用と Perch v2 のネイティブ予測用とで別々にチューニングしました**。どちらも submission を反復するのではなく、上記の検証スプリット上でチューニングしています。

ファインチューニング済みモデル

1. **LSS からの Site priors (サイト事前分布)**。site のみの組み合わせ、 $\lambda = 0.5$ 、logit 空間で適用し、そのサイトで出現することが分かっている種をブーストする — 共有ノートブックから取り入れたアイデア。
2. **Max sample/window boost**。各チャンクを、そのクラスのファイルごとの最大確率に向けてブーストする ($\alpha = 0.20$)。
3. **Label smoothing**。クラス条件付きカーネルを用いた時間方向のガウシアン smoothing: Amphibia/Insecta にはよりフラットな $[0.33, 0.33, 0.33]$ カーネルを、その他のラベルにはより鋭い $[0.2, 0.6, 0.2]$ カーネルを使用。
4. **Genus/class taxonomy smoothing (属/クラスの分類階層 smoothing)**。同じ genus の種 ($\alpha = 0.15$) と同じ class の種 ($\alpha = 0.05$) を smoothing する — 共有ノートブックから取り入れたアイデア。
5. **Delta TTA**。プーリングウィンドウを2フレームずらしてフレーム単位の予測の max をブレンドする (SED のみ)。

Perch v2 (ネイティブ予測)

1. **Window-blend smoothing**。時間方向の **5-chunks-window** ($\alpha = 0.7$) と **12-chunks-window** ($\alpha = 0.05$) の max 値のブレンド。
2. **Genus/class taxonomy smoothing**。同じ genus の種 ($\alpha = 0.15$) と同じ class の種 ($\alpha = 0.05$) を smoothing する。

推論とアンサンブル

驚くべきことに、ファインチューニング済みモデルと比べて Perch のスコアが平凡 (LSS と重複する種で ~ 0.9) であるにもかかわらず、そのネイティブ予測はアンサンブルにとって不可欠でした。ブレンドを試して LB に提出する中で、Perch のブーストの大部分が非 LSS 種から来ていることに気づきました。それらの種は、テストドメインの ground truth がなかったため、おそらく Perch のスコアに近い値になっており、その識別の多くを Perch が担っていたのです。

最終予測は、ファインチューニング済みアンサンブルと Perch v2 のネイティブ線形ヘッド予測の重み付きブレンドです。2つは異なるスコア分布を持つ異なるパイプラインから来ているため、生の確率を平均するのではなく、ブレンド前に各クラス列を rank-transform (ランク変換) します。

- ファインチューニング済みアンサンブル (SED + MLP + スペシャリスト) : 0.8
- Perch v2 native: 0.2

ヘッドが制限されたモデル (genus および Amphibia/Insecta スペシャリスト) は、より狭いラベル空間上で予測を行い、ブレンドに入る前にマスクを用いて全234クラス幅に散らし戻されます。

参考文献

1. van Merriënboer, B., Dumoulin, V., Hamer, J., Harrell, L., Burns, A., & Denton, T. (2025). Perch 2.0: The Bittern Lesson for Bioacoustics. arXiv. <https://doi.org/10.48550/arXiv.2508.04665>
2. Heinrich, R., Rauch, L., Sick, B., & Scholz, C. (2025). AudioProtoPNet: An interpretable deep learning model for bird sound classification. Ecological Informatics, 87, 103081. <https://doi.org/10.1016/j.ecoinf.2025.103081>

3. Nikita Babych. (2025). 1st Place Solution: Multi-Iterative Noisy Student Is All You Need. Kaggle. <https://doi.org/10.34740/KAGGLE/W/12619>

リソース

- Perch v2 のモデル重み (Kaggle) : <https://www.kaggle.com/models/google/bird-vocalization-classifier>
- AudioProtoPNet-5-BirdSet-XCL (Hugging Face) : <https://huggingface.co/DBD-research-group/AudioProtoPNet-5-BirdSet-XCL>
- 推論ノートブック: <https://www.kaggle.com/code/nikitababich/birdclef2026-1st-place-inference>
- ベストアンサンブルモデル: <https://www.kaggle.com/datasets/nikitababich/birdclef-2026-1st-place-models>
- 推論ソースコード: <https://www.kaggle.com/datasets/nikitababich/birdclef-2026-1st-place-src-inference>

補足Q&A

質問者 (Cody_Null) : 2連続の1位獲得なんて本当に信じられません、おめでとうございます！ データ処理/学習のパイプラインを共有する予定はありますか。自分がどこで道を誤ったのかをもっと知りたいのです。私は今年このコンペにあまり時間をかけられませんでした。自分のカスタム学習パイプラインの多くの部分で、あなたと似たアイデアを持っていたようなので、それらが学習の中でどう結びついていたのか興味があります。

著者 (Nikita Babych) : ありがとうございます。ちょうど推論のソースコードを共有したところで、そこにはすでに私の解法の大部分の要素（アーキテクチャ、前処理、後処理など）が含まれています。残りの部分についても、専用のリポジトリを作成して共有できればと思っています。

推論 src: <https://www.kaggle.com/datasets/nikitababich/birdclef-2026-1st-place-src-inference>